

Teste de Software

Prof. Dr. Bruno Queiroz Pinto

Testes Automatizados Spring Boot

Baixem o projeto modelo : Repositório GitHub

- ✓ <https://github.com/brunoqp78/projeto-veterinario-spring-boot-h2>
 - Clonem ou baixem o projeto disponibilizado no GitHub.

Testes Automatizados em Java

Nosso objetivo é o teste

- Projeto implementa uma Rest API;
- Projeto Monolítico utilizando Thymeleaf;
- Utiliza o JPA para o Spring boot;
- Utiliza o banco de dados de teste H2;
- Implementa os serviços de acesso aos dados de um veterinário.
- Utiliza as classes Optional, Page incorporada no JPA

<https://www.baeldung.com/java-optional-uses>

<https://docs.spring.io/spring-data/commons/docs/current/api/org/springframework/data/domain/Page.html>

Testes Automatizados em Java

Nosso objetivo é o teste

- Arquitetura API Rest desse projeto:
 - ✓ Controller : implementa o controller, faz o mapeamento.
 - ✓ Services : implementa as regras de negócio que serão servidas.
 - ✓ Repositories : repositórios necessários para utilizar o spring data jpa. Encapsula as operações de acesso aos dados.
 - ✓ Entities : são os objetos que persistem no banco de dados.
 - ✓ ~~Dto : objetos utilizados para transferência de dados.~~

Explicação DTO : [Link](#)

Testes Automatizados em Java

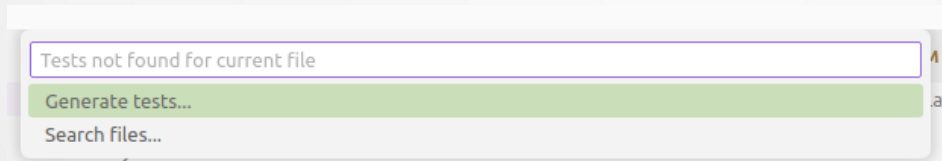
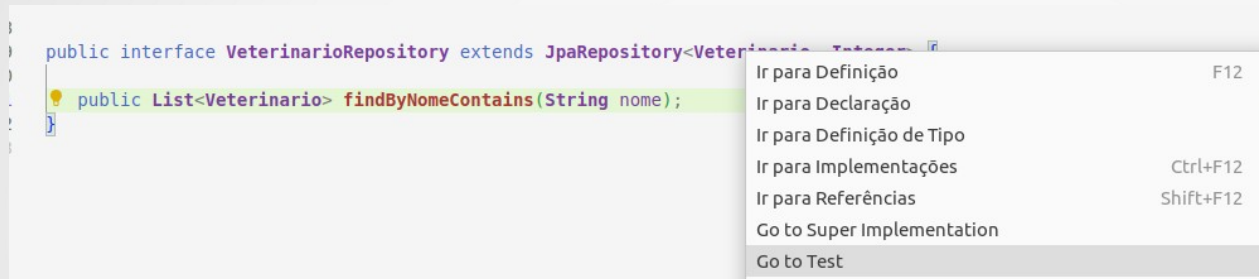
- Primeiro teste:
 - ✓ Testar a classe VeterinarioRepository
 - ✓ Criar a classe de teste VeterinarioRepositoryTest em:
src/test/java/com/iftm/gerenciadorveterinarios/**tests/repository**

gerenciador-Veterinarios > src > test > java > org > iftm > gerenciadorveterinarios > repository >  VeterinarioRepositoryTest.java > ...

```
1 package org.iftm.gerenciadorveterinarios.repository;
2
3 public class VeterinarioRepositoryTest {
4 }
5 |
```

Testes Automatizados em Java

- No VSCode:



Não precisa marcar, por enquanto,
o método existente.

Testes Automatizados em Java

- Primeiro teste:
 - ✓ Precisamos carregar um contexto para o teste.
 - ✓ Inserir a notação `@DataJpaTest`: Carrega somente os componentes relacionados ao Spring Data JPA. Cada teste é transacional e dá rollback ao final.

```
package org.iftm.gerenciadorveterinarios.repository;

import static org.junit.jupiter.api.Assertions.fail;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;

@DataJpaTest
public class VeterinarioRepositoryTest {

    @Test
    void test() {
        fail("Not yet implemented");
    }
}
```

Testes Automatizados em Java

- Primeiro teste:
 - ✓ Fazer injeção de dependência da classe a ser testada.
 - `private VeterinarioRepository repository;`

```
@DataJpaTest
class VeterinarioRepositoryTest {
    @Autowired
    private VeterinarioRepository repositorio;

    @Test
    void test() {
        fail("Not yet implemented");
    }
}
```


Cenário de teste 01

- Objetivo: Verificar se a busca por id realmente retorna o veterinário correto.
- monta o cenário,
 - arquivo import.sql carrega o cenário (veterinários cadastrados)
 - definir o id de um veterinário que exista em import.sql

```
INSERT INTO tb_veterinario (nome, email, especialidade, salario) VALUES('Conceição Evaristo', 'conceicao@gmail.com', 'pequenos', 3500.0);  
INSERT INTO tb_veterinario (nome, email, especialidade, salario) VALUES('Erica Queiroz Pinto', 'erica@gmail.com', 'grandes', 4500.0);
```

O Id é auto-incremental, então Conceição é 1
e Erica é 2.

Cenário de teste 01

- executa a ação
 - executar o método de buscar por id
- e valida a saída.
 - verifica se foi retornado um objeto.
 - verifica se o nome e o email do veterinário retornado corresponde ao esperado.

Testes Automatizados em Java

- Primeiro define o id a ser buscado, então executa o método `getById` que deve retornar um objeto `Veterinario`.
- Então fazemos uma verificação se o objeto foi retornado.

```
/**
 * <p><b>Cenário:</b> Verificar se a busca por id realmente retorna o veterinario correto.</p>
 * <p><b>Entrada:</b> idExistente = 1 </p>
 * <p><b>Saída esperada:</b> Veterinário com nome "Conceição Evaristo" e email 'conceicao@gmail.com' </p>
 */
@Test
@DisplayName("Testa se a busca pelo ID realmente retorna o veterinario correto.")
public void testarBuscaPorIdRetornaVeterinarioCorreto(){
    //arrange
    Integer idexistente = 1;
    String nomeEsperado = "Conceição Evaristo";
    String emailEsperado = "conceicao@gmail.com";
    //Act
    Veterinario veterinarioObtido = repositorio.getById(idexistente);
    //Assert
    assertNotNull(veterinarioObtido);
    assertEquals(nomeEsperado, veterinarioObtido.getNome());
    assertEquals(emailEsperado, veterinarioObtido.getEmail());
}
```



Lombok Annotations Support for VS Code

Microsoft microsoft.com | 1,020,853 | ★★★★★ (19)

Refactor code with Lombok annotations, or remove Lombok annotations with actual methods.

[Disable](#)

[Uninstall](#)

[Switch to Pre-Release Version](#)

☒ Auto Update

Testes Automatizados em Java

- Outra versão, utilizando a classe Optional:

```
@Test
@DisplayName("Testa se o método findById realmente retorna o veterinário correto.")
void testaSeBuscaVeterinarioEncapsuladoQuandoExisteId() {
    Integer idExistente = 1;
    String nomeEsperado = "Conceição Evaristo";
    String emailEsperado = "conceicao@gmail.com";

    Optional<Veterinario> resultado = repositorio.findById(idExistente);
    assertTrue(resultado.isPresent(), "Objeto não encontrado.");
    assertEquals(nomeEsperado, resultado.get().getNome());
    assertEquals(emailEsperado, resultado.get().getEmail());
}
```

Testes Automatizados em Java

- Optional : é uma classe que encapsula um objeto evitando uma resposta que seja Nula. (Optional é utilizado no JPA Spring)
- Os métodos findById e getById são padrões do JPA ou pode ser reescrito via @Query, como o exemplo acima que deve ser inserido na classe VeterinarioRepository.

Versão com erro

```
@Query("SELECT v FROM Veterinario AS v WHERE v.id = 2")  
public @NonNull Optional<Veterinario> findById(@NonNull Integer id);
```

```
@Query("SELECT v FROM Veterinario AS v WHERE v.id = 2")  
public @NonNull Veterinario getById(@NonNull Integer id);
```

Versões corretas:

```
@Query("SELECT v FROM Veterinario AS v WHERE v.id = :id")  
public @NonNull Optional<Veterinario> findById(@NonNull Integer id);
```

```
@Query("SELECT v FROM Veterinario AS v WHERE v.id = :id")  
public @NonNull Veterinario getById(@NonNull Integer id);
```

Cenário de teste 02

- Objetivo: Verificar se a busca por id inexistente retorna nenhum veterinário.
- monta o cenário,
 - arquivo import.sql carrega o cenário (veterinários cadastrados)
 - definir o id de um veterinário que não exista em import.sql
- executa a ação
 - executar o método de buscar por id
- e valida a saída.
 - verifica se não foi retornado um objeto.

Cenário de teste 03

- Objetivo: Verificar se a exclusão realmente apaga um registro existente.
- monta o cenário,
 - arquivo import.sql carrega o cenário (veterinarios cadastrados)
 - definir o id de um veterinario que exista em import.sql
- executa a ação
 - executar o método de exclusão por id
 - executar o método de buscar por id
- e valida a saída.
 - verificar se o resultado do método de busca é falso ou nulo.

Testes Automatizados em Java

- Primeiro define o id a ser buscado, então executa o método `deleteById` que deve apagar o veterinario do BD.
- Por fim, executamos o método `findById` para buscar o objeto com determinado ID. O método `deleteById` funciona se realmente o veterinário foi apagado.

```
@Test
@DisplayName("Testa se o método deleteById realmente apaga o veterinário correto.")
void testaSeApagaVeterinarioEncapsuladoExisteId() {
    Integer idExistente = 1;
    int numeroElementosExistentes = 1;
    repositorio.deleteById(idExistente);
    Optional<Veterinario> resultado = repositorio.findById(idExistente);
    assertFalse(resultado.isPresent(), "Objeto encontrado");
    assertEquals(numeroElementosExistentes, repositorio.count(),
        "Numero de elementos do Banco de dados incorretos");
}
```


Testes Automatizados em Java

- O objeto Veterinario não deve estar presente no Optional resultado.
- Implementações
 - `@Modifying` permite modificar Comandos de manipulação do JPA

Versão com erro

```
@Modifying
@Query("DELETE FROM Veterinario v WHERE v.id >= :id")
public void deleteById(@NonNull Integer id);
```

Versões corretas:

```
@Modifying
@Query("DELETE FROM Veterinario v WHERE v.id = :id")
public void deleteById(@NonNull Integer id);
```

//Ou

```
public void deleteById(@NonNull Integer id);
```

Cenário de teste 04

- Objetivo: Verificar se a exclusão retorna um erro quando um id não existente é informado.
- monta o cenário,
 - arquivo import.sql carrega o cenário (veterinarios cadastrados)
 - definir o id de um veterinario que não exista em import.sql
- executa a ação
 - executar o método de exclusão por id
- e valida a saída.
 - verificar se ocorre o erro:
EmptyResultDataAccessException

Testes Automatizados em Java

- Primeiro define o id a ser buscado, então executa o método deleteById que gerar uma exception da classe EmptyResultDataAccessException.

```
@Test
void testaSeApagarObjetoQuandoNaoExisteIdRetornaException() {
    long idExistente = 10000;
    Assertions.assertThrows(EmptyResultDataAccessException.class,
        () -> { repositorio.deleteById(idExistente); });
}
```

Testes Automatizados em Java

- I have a problem → código desenvolvido não retorna o erro.

Versões corretas:

```
@Modifying
@Query("DELETE FROM Veterinario v WHERE v.id = :id")
public void deleteById(Integer id);
```

- Solução utilizar apenas Query Methods do JPA, apagando essas linhas, ou criar uma interface e uma implementação que gere a exception. (Bem complicado e desnecessário)

Versão apagando o método

```
@Repository
public interface VeterinarioRepository extends JpaRepository<Veterinario, Integer>, VeterinarioRepositoryCustom {

}
```

Testes Automatizados em Java

- Exemplo do complicado (extra – não testei no veterinário)

```
@Repository
public interface VeterinarioRepository extends JpaRepository<Veterinario, Integer>,
VeterinarioRepositoryCustom {

    @Modifying
    @Query("DELETE FROM Veterinario c WHERE c.id = :id")
    public void deleteByIdOriginal(Integer id);

}

interface VeterinarioRepositoryCustom {
    void deleteById(Integer id);
}

class VeterinarioRepositoryImpl implements VeterinarioRepositoryCustom {
    @Autowired
    VeterinarioRepository orderRepository;

    @Override
    public void deleteById(Integer id) {
        long elementoI = orderRepository.count();
        orderRepository.deleteByIdOriginal(id);
        long elementoF = orderRepository.count();
        if (elementoF==elementoI) {
            throw new EmptyResultDataAccessException((int)elementoI-1);
        }
    }
}
```

Cenário de teste 05

- Objetivo: Verificar se a exclusão de todos elementos realmente apaga todos os registros do Banco de dados.
- monta o cenário,
 - arquivo import.sql carrega o cenário (veterinário cadastrados)
- executa a ação
 - executar o método de exclusão de todos registros
- e valida a saída.
 - consultar todos os registros do banco e verificar se retorna vazio.

Testes Automatizados em Java

- Primeiro chama o método `deleteAll` e então chama o método `count` para verificar quantos elementos há.

```
@Test
void testaSeApagarTodosTornaBDVazio() {
    repositorio.deleteAll();
    assertEquals(repositorio.count(), 0);
}
```

Cenário de teste 06

- Objetivo: Verificar se a exclusão de uma entidade existente no banco de dados realmente ocorre.
- monta o cenário,
 - arquivo import.sql carrega o cenário (veterinários cadastrados)
 - id de um veterinário que existe em import.sql
- executa a ação
 - executa o método encontrar por id para retornar a entidade do veterinário com id informado.
 - executa o método apagar por id
 - executar novamente o método encontrar por id e verificar se o retorno dele é vazio
- e valida a saída.
 - verifica se o retorno do método encontrar por id é vazio.

Testes Automatizados em Java

- Definir um ID que existe, exemplo = 1. Chamar o método findById que retorna um Veterinários se houver Id. Utilizar o método delete para apagar o veterinário retornado e então utilizar o método findById novamente para verificar se o veterinário realmente foi apagado.

@Test

```
void testaSeApagarTodosTornaBDVazio2() {  
    long idExistente = 1;  
    Optional<Veterinario> resultado = repositorio.findById(idExistente);  
    repositorio.delete(resultado.get());  
    Assertions.assertTrue(repositorio.findById(idExistente).isEmpty());  
}
```

Cenário de teste 07

- Objetivo: Verificar se um veterinário pode ser excluído pelo email.
- monta o cenário,
 - arquivo import.sql carrega o cenário (veterinarios cadastrados)
 - e-mail de um veterinário cadastrado
- executa a ação
 - executar um método para excluir um veterinário pelo email (não existe ainda).
 - buscar um veterinário pelo email (não existe)
- e valida a saída.
 - a busca deve retornar vazia.

Testes Automatizados em Java

- Primeiro chama o método delete para apagar o registro pelo email informado, implementar o método delete **VeterinarioByEmail**.
- Então verifica se o método **findVeterinarioByEmail** retorna vazio.

```
@Test
void testaSeApagarVeterinarioPeloEmail() {
    repositorio.deleteVeterinarioByEmail("conceicao@gmail.com");
    Optional<Veterinario> resultado = repositorio.findVeterinarioByEmail("conceicao@gmail.com");
    Assertions.assertTrue(resultado.isEmpty());
}
```

Testes Automatizados em Java

- Utilizando TDD iríamos implementar os métodos:

```
@Repository
public interface VeterinarioRepository extends JpaRepository<Veterinario, Integer> {

    @Modifying(clearAutomatically = true)
    @Query("DELETE FROM Veterinario obj WHERE "
            + "obj.email = :email")
    void deleteVeterinarioByEmail(String email);

    @Query("SELECT DISTINCT obj FROM Veterinario obj WHERE "
            + "obj.email = :email")
    Optional<Veterinario> findVeterinarioByEmail(String email);
}
```